

PROCEDURE:

ASTRAZIONE DEL CODICE: (Variabili Lo Sono Già)

- /// Ignorare dettagli non rilevanti
- /// Identificare PROPRIETÀ D'INTERESSE

astrazione della cella di MEMORIA ←

INTERFACCIA ⇒ Input Richiesto e OUTPUT dato

Anche La Classe è un Astrazione (delle dichiarazioni)

All' Astrazione **si dà un Nome**, per poi Richiamarla

Parametri rendono il CODICE in Grado di EFFETTUARE molte **Computazioni diverse**.

Permettono di Rappresentare con un UNICO CODICE una Funzione dipendente da dei valori di INPUT

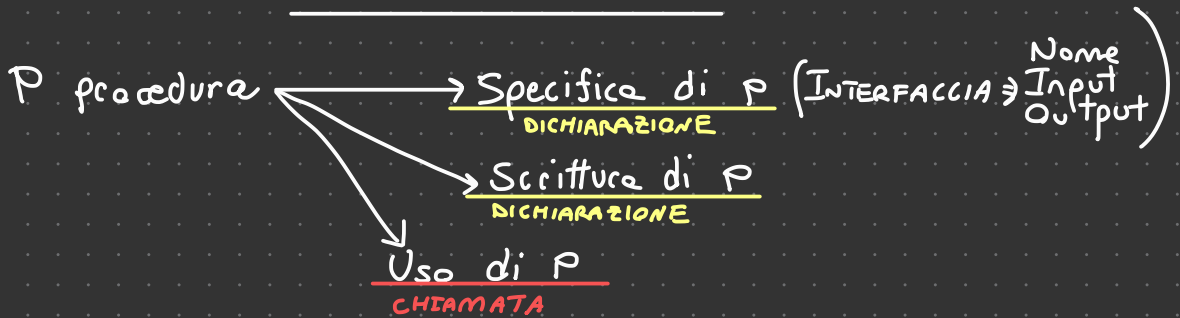
Corpo che è **l'INSIEME delle ISTRUZIONI** che Vengono eseguite quando si RIFERISCE il NOME delle procedure.

È la parte INCAPSULATA NELLA PROCEDURA... non serve a chi la Chiama.

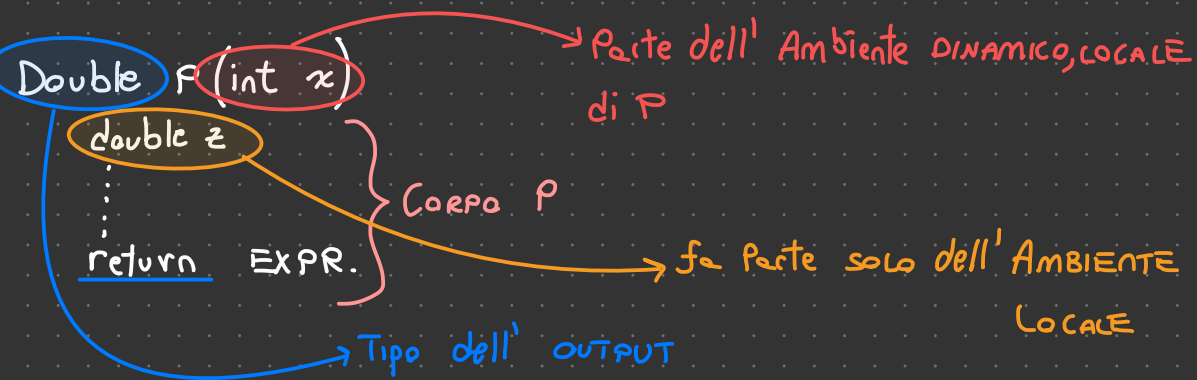
In Generale la PROCEDURA ha una SOLA ENTRATA (una Sorte di Attivazione Annidata ma da un'altra parte con le Regole di Visibilità dei BLOCCHI ANNIDATI)

L'EXEC del Codice ChiamaNte Viene SOSPESA per Passare il controllo alla procedura chiamata

Al Termine della procedura il Controllo TORNA Sempre al chiamaNte che RIPRENDE L'EXEC. (un PROCESSO AUTOMATICO)



Mi Servono STRUMENTI SINTATICI per Questi STEP



Interfaccia è: `DOUBLE P (int x)` che raccoglie TUTTE le info utili per Sfruttare la Procedura

Uso Di P $\Rightarrow$ è la chiamata

main
double y;
y = P(3);

Parametro Attuale $\leftarrow$

VISIBILITÀ

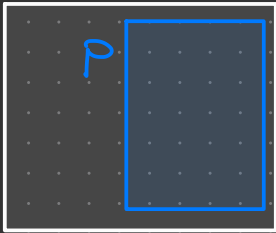
main
int x
P(x+1)

void P(int y) {
z = x + y;
}

non definita DENTRO P

Quale valore uso per x?

MAIN



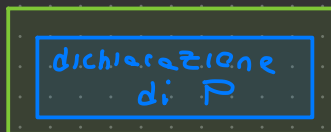
Questa è la REGOLA di VISIBILITÀ, come se fosse un BLOCCO Annidato

Regole di visibilità Sono NECESSARIE per risolvere i Riferimenti Non Locali $\Rightarrow$ Rimangono le STESS: ogni dichiarazione è VISIBILE nel Blocco in cui è DEFINITA e nei BLOCCHI

annidati che non la Sovrascrivono

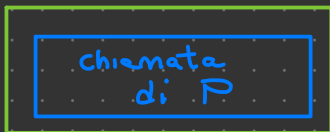
Apparentemente non cambia ma così abbiamo 2 tipi di AMBIAMENTI

/// STATICO → Annidamento della definizione del Blocco, Quindi dove si Trova la DICHIARAZIONE



AMBIENTE DI DICHIARAZIONE

/// DINAMICO → Annidamento della chiamata e quindi dell'EXEC del Blocco quindi dove si Trova la chiamata



AMBIENTE DI CHIAMATA

Programmatore deve conoscere Queste 2 Regole di SCOPING (dinamico/Statico)

REGOLE PER DETERMINARE L'AMBIENTE DI RIFERIMENTO

- /// Di Scope → Quando l'AMBIENTE NON è LOCALE
- /// Di Parametri →
- /// Di Binding →

REGOLE DI SCOPE:

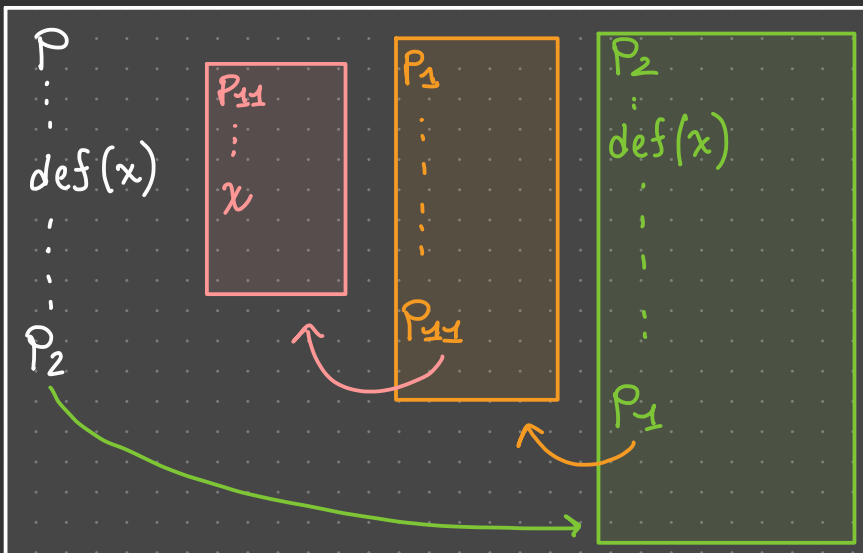
```
{ int x = 10  
  void foo() {  
    x++;  
  }  
  void fee() {  
    int x = 0;  
    foo();  
  }  
  fee();  
}
```

x non locale, Ma dove Trova Significato?
Alla Riga Sopra, dove contiene la definizione
che è **STATICA** o **DINAMICO**, dove contiene la
Chiamata

Regole di Scoping
Mi definisce Quale dei 2 devo Usare

Con Ambiente Dinamico può Potenzialmente CAMBIARE per
ogni CHIAMATA, altrimenti è SEMPRE UGUALE visto che non
dipende dalla CHIAMATA

SCOPING STATICO:



SEQUENZA CHIAMATE: $P \rightarrow P_2 \rightarrow P_1 \rightarrow P_{11}$

SCHEMA DEFINIZIONI: $P \rightarrow P_1 \rightarrow P_2 \rightarrow P_{11}$
USO x

↓
della CATENA Statica

↳ potrei risalire all'INDIETRO da P_{11} raccogliendo delle INFO a Tempo di Compilazione per Non Sprecare Tempo POI

```
{ int  $\otimes x = 0$ ;  
  void pippo(int m) {  
     $\otimes x := m + 1$ ;  
  }  
  pippo(3);  
  write( $\otimes x$ );  
  { int x = 0;  
    pippo(3);  
    write(x);  
  }  
  write(x);  
}
```

NON LOCALE, fa sempre riferimento alla x Globale della Riga Sopra

→ Modifica la x Globale $\otimes$ a 4

→ Scrive 4 riferendosi a $\otimes$

→ Viene Assegnato il val 4 alla Variabile Globale

→ Scrive $\otimes$ riferendosi Alla x del Blocco

→ Se fosse Stato un Valore $\neq$ Allora anche la WRITE di fuori Sarebbe $\neq$

Nello SCOPING Statico l'AMBIENTE è INDIPENDENTE dalla Posizione della Chiamata

NO

$x = 1;$

$\text{int } x;$

SI

$\text{void } f() \{$

$g();$

$\}$

$\text{void } g() \{$

$f();$

$\}$

INDIPENDENZA DALLA POSIZIONE DELLA CHIAMATA, È MENO

ADATTABILE/FLESSIBILE

Python e Javascript Sono degli Esempi

SCOPING DINAMICO:

Stabilisce l'AMBIENTE a partire dalla Seq. di Chiamate:

SEQUENZA CHIAMATE: $P \xrightarrow{\text{green}} P_2 \xrightarrow{\text{orange}} P_1 \xrightarrow{\text{pink}} P_{11}$ dove uso x

Che Viene detta Catena dinamica

```
{ int  $\hat{x}$  = 0;
  void pippo(int m) {
     $x := m + 1$ 
  }
  pippo(3);  $\rightarrow$  Porta  $x$  Globale * a 4
  write( $x$ );  $\rightarrow$  4
  { int  $\hat{x}$  = 0;
    pippo(2);  $\rightarrow$  Porta  $\hat{x}$  Locale a 3
    write( $x$ );  $\rightarrow$  3
  }
  write( $x$ );  $\rightarrow$  4
}
```